MLR-X

A Scalable Software for Multiple Linear Regression in Low- and High-Dimensional Data

User Guide

Version 1.0



Homepage:

https://jacksonalcazar.github.io/MLR-X

Jackson J. Alcázar

jjalcazar.dev@gmail.com

Santiago, Chile.

This document describes the operational use of the MLR-X desktop application for data preparation, multiple linear regression (MLR) model search, validation, diagnostics, visualization, reporting, and command-line automation.

Contents

1 Overview **3**

2 Distribution and installation modes **3**

2.1 Platform Support . . . 3

2.2 Interface Variants . . . 3

2.2.1 GUI-Only Versions . . . 3

2.2.2 GUI+CLI Versions . . . 3

2.2.3 Linux Behavior . . . 3

3 Requirements and installation **4**

3.1 Portable package mode (no Python required) . . . 4

3.2 Setting up the portable packages . . . 4

3.3 Python package mode (`pip install mlr-x`) . . . 5

4 Interface (GUI) tour **6**

4.1 **Setup & Run** tab . . . 6

4.2 **Models** tab . . . 7

4.3 **Validation** tab . . . 8

4.4 **Diagnostics** tab . . . 9

4.5 **Visualization** tab . . . 9

4.6 **Summary** tab . . . 9

4.7 **Prediction** tab . . . 10

4.8 **Desktop** tab (Variable Explorer) . . . 11

4.9 **Help** tab . . . 11

5 Analysis flow and process handling **11**

5.1 Before starting . . . 11

5.2 Starting a run . . . 11

5.3 Progress and cancellation . . . 12

5.4 Completion and outputs . . . 12

6 Search algorithms **12**

6.1 EPR-C3 (Expand–Perturb–Reduce–C3) . . . 12

6.2 All-subsets (traditional exhaustive search) . . . 13

7 Summary and reports **13**

8 Command-line (CLI) usage **14**

8.1 Invocation . . . 14

8.2 Configuration file 14
8.3 Optional arguments 14
8.4 Output 15
8.4.1 Diagnostics — Tables 15
8.4.2 Visualization — Charts 15
8.4.3 Summary 16
8.5 CLI execution example 16

9 Operational good practices **16**

10 Configuration Parameters, Metrics, and Diagnostic Charts **17**
10.1 Key configuration parameters 17
10.2 Metric definitions 19
10.3 Interpreting key charts 25
10.4 Applicability Domain (AD) 27

11 Troubleshooting and FAQ **28**

12 License, credits, and citation **29**

13 Configuration file example **29**

1 Overview

MLR-X is a cross-platform application for designing, fitting, and assessing multiple linear regression models on datasets that range from small matrices to high-dimensional tables. The software combines a graphical user interface (GUI) with optional command line interface (CLI) execution, supports both heuristic and exhaustive model searches, and delivers synchronized validation, diagnostics, visualization, prediction, and reporting outputs in portable CSV tables, TXT summaries, and figure exports (PNG, TIFF, PDF, SVG). The prediction workflow includes applicability-domain interpretation with robust handling of outside-domain cases and missing predictor values.

2 Distribution and installation modes

MLR-X is distributed in two complementary modes:

- **Portable desktop packages**: self-contained executables intended for users who do not want to manage Python environments.
- **Python package (`pip install mlr-x`)**: a Python-based installation intended for scripted workflows and terminal-first usage.

2.1 Platform Support

- **Linux**: Single binary executable
- **macOS**: Application bundles (.app)
- **Windows**: Portable executables (.exe)

2.2 Interface Variants

2.2.1 GUI-Only Versions

- Launch only the graphical user interface
- Do not display a terminal or console window
- Available for macOS (.app) and Windows (.exe)

2.2.2 GUI+CLI Versions

- Launch both the graphical interface and a terminal or console
- Allow simultaneous GUI and command-line interaction
- Keep the terminal open for CLI commands alongside the GUI
- Available for macOS (.app) and Windows (.exe)

2.2.3 Linux Behavior

- A single binary supports both modes

- Launches the GUI-only mode by default
- Provides CLI access via command-line arguments
- Does not require a separate GUI+CLI variant

CLI-enabled platforms allow running the full pipeline without the GUI by supplying a `config.conf` file.

3 Requirements and installation

3.1 Portable package mode (no Python required)

The portable packages are self-contained downloads and do not require Python, virtual environments, or scientific-library setup. Place each executable in a preferred folder. Optionally, add that folder to the system `PATH` so `MLR-X` can launch from any directory (e.g., `export PATH=/opt/MLRX:$PATH` on Linux or configuring environment variables on Windows). Choose the appropriate package and follow these steps:

Supported platforms and architectures:

- Windows 10/11 (64-bit)
- macOS X (Arm64)
- Ubuntu 20.04 (x86-64)

3.2 Setting up the portable packages

- **Windows GUI**: download the GUI package, place `MLRX.exe` in a preferred folder, and run it directly.
- **Windows GUI+CLI**: same as Windows GUI, with the Run CLI option enabled.
- **macOS GUI**: download the app bundle and run it from a preferred location (e.g., move it to `/Applications` if desired).
- **macOS GUI+CLI**: same as macOS GUI, with the Run CLI option enabled.
- **Linux**: before running MLR-X, install the required system packages:

```
sudo apt-get install python3-tk
sudo apt-get install xvfb
```

Then download the binary, move it to a stable folder, and mark it as executable if needed.

1. Create a stable installation directory and copy the executable (and icon, if desired).

```
sudo mkdir -p /opt/MLRX
sudo cp MLRX /opt/MLRX/
sudo chmod +x /opt/MLRX/MLRX

sudo mkdir -p /opt/MLRX/icon
```

```
sudo cp icon/icon.ico /opt/MLRX/
```

2. (Optional) Add the installation directory to the `PATH` (recommended for CLI usage).

```
# For the current session
export PATH=/opt/MLRX:$PATH

# Make permanent (choose one, depending on your shell)
echo 'export PATH=/opt/MLRX:$PATH' >> ~/.bashrc
# or
echo 'export PATH=/opt/MLRX:$PATH' >> ~/.zshrc
```

3. (Optional) Register a desktop launcher. The package includes `MLRX.desktop`. Update its `Exec` and `Icon` fields to match the installation paths (if necessary) using a text editor, then follow the next steps:

```
chmod +x MLRX.desktop
cp MLRX.desktop ~/.local/share/applications/
```

4. (Optional) Refresh the application database (only if the command is available on the distribution).

```
update-desktop-database ~/.local/share/applications 2>/dev/null || true
```

3.3 Python package mode (`pip install mlr-x`)

Use this mode when MLR-X is installed from PyPI and executed from a Python environment.

Prerequisite: Python 3.10+ is recommended.

1. (Recommended) Create and activate a virtual environment:

```
# Linux / macOS
python3 -m venv .venv
source .venv/bin/activate

# Windows (cmd)
python -m venv .venv
.venv\Scripts\Activate
```

2. Install or upgrade the package:

```
python -m pip install --upgrade pip
pip install mlr-x
```

3. Launch MLR-X (GUI):

```
mlrx
```

4. Optional verification:

```
pip show mlr-x
mlrx --help
```

Notes

- Portable and pip-based installations are alternative deployment modes; users generally need only one of them.

4 Interface (GUI) tour

The main window presents a tabbed workflow.

4.1 Setup & Run tab

This tab manages datasets, model search settings, and execution. A progress bar tracks the active task, while a status bar shows current task status. Additional tabs become available automatically once their prerequisites are completed.

Dataset

1. Choose the CSV via the field or *Browse* button.
2. Set the delimiter (`;` by default), target (dependent variable), and any non-predictor columns.
3. Optionally exclude observations by ID or ranges (e.g., `3,9,4-8`).
4. Enable *Exclude nearly constant predictors* and provide the percentage threshold to drop near-zero-variance variables.
5. Actions: *Preview data*, *Run analysis*, *Cancel*. Status appears in the upper-right corner; the active task shows in the lower-right footer.

Data splitting Choose how to partition the dataset for training and testing.

- *None*: use the full dataset.
- *Random*: specify *Test size (%)*.
- *Manual*: specify *training IDs* and *test IDs* without overlap.

Selecting *None* keeps external validation available later from the Validation tab by loading a separate file.

Settings

1. Search method: **All Subsets** (default) or **EPR-C3** (Expand–Perturb–Reduce–C3). For further details see section 6.

2. EPR-C3 parameters: *Number of seeds*, *Seed size*, and *Set iterations/seed*. Seeds are independent starting points for exploring model combinations; defaults are provided for a balanced start. Iterations per seed can run in *auto* (estimate and apply a maximum), *manual* (enter a maximum), or *converge* (run until convergence). The *Seed settings* dialog consolidates EPR-C3 controls into three sections:

   - Seed size: choose the default size (kept synchronized at half the *Max predictors per model*) or apply a custom size. A *Changes applied!* label appears after updating.
   - Reproducibility: set *Random state* (default 42) to make EPR-C3 seed generation reproducible. A *Changes applied!* label confirms the update.
   - Guardrail: enable or disable the *Allow fewer than 1000 seeds* checkbox. Disabling it keeps the minimum at 1,000 seeds; enabling it allows smaller runs with a confirmation. A *Changes applied!* label confirms the update.

3. Key parameters: *Max predictors per model*, *Significance level*, *Correlation threshold*, *VIF threshold*, *Target metric cutoff*, *Top models to report*, and *CPU cores to use* (for further details, see Section 10.1). *Top models to report* defines the maximum number of top-performing models that are exported and used in CSV files, diagnostic tables, visualizations, and summaries (default: 5000). Predictors correlation and Variance Inflation Factor (VIF) thresholds gate multicollinearity; the Target metric cutoff uses the training-set value of the selected objective as a gatekeeper even when optimizing alternative metrics.

4. Target metric and covariance: select R^2, adj-R^2, Q^2 (LOO), RMSE (LOO), or CCC as the scoring objective and pick the covariance estimator (Non-robust, HC0–HC3), for further details, see Section 10.1.

5. Clip predictions to: set lower and/or upper bounds to constrain predicted values for scoring and exports.

If the chosen configuration is outside recommended thresholds, the GUI opens a custom warning dialog offering *Continue* or *Cancel analysis* so you can explicitly proceed or abort.

4.2 Models tab

Displays three synchronized result tables: training, internal validation, and external validation. The header controls include *Browse* and *Load* to open model result files, *Clear* to remove the results from the interface, and *Filter* to filter models by predictor composition, model IDs, and maximum VIF. Sorting is applied independently in each table by double-clicking column headers (left or right click). A left double-click on a model row opens a per-model preview (Predicted vs Observed plot for the selected model). A right-click on that row opens a context menu with direct navigation shortcuts to *Diagnostics*, *Visualization*, and *Summary*, plus *Select and Push* to mark that model as the default active model.

The *Filter models* dialog provides the following optional criteria:

- **Model IDs (1, 3)**: enter one positive integer ID or several IDs separated by commas, for example `1, 3, 7`. Spaces are accepted and repeated IDs are ignored. A model passes this criterion if its ID is one of the entered IDs. Ranges such as `1-5` are not supported in this dialog.
- **VIF max** $\leq$: below the model-ID field, enter a finite, non-negative number in the compact input field. Only models whose training-table VIFmax is less than or equal to this limit are retained; equality is included. Models with a missing or non-finite VIFmax do not pass an active VIF filter.
- **Filter by variable**: enter a predictor name and click *Add*. Up to six variables can be listed; use their checkboxes to activate them or *Remove* to remove them. Retained models must contain all checked predictors; name matching is case-insensitive.

Leave either numeric field blank to ignore that criterion. Click *Apply* to combine all active criteria: for example, IDs `1, 3, 7`, a VIFmax limit of `5`, and a checked predictor `x` retain only models with one of those IDs, VIFmax ≤ 5, and predictor `x`. Filtering uses the complete loaded model set, so subsequent changes are not restricted to the previous filtered subset. Clear both numeric fields and uncheck or remove all variables, then click *Apply* to restore all loaded models. Invalid entries produce a warning. If no model matches, an informational message is shown and the current results remain unchanged.

All tables contain the predictor set and the predictor count (*Size*). Each table then reports the metrics corresponding to its evaluation stage:

- **Top models training**: goodness-of-fit metrics (R^2, adjusted R^2, RMSE, MAE, s), multicollinearity (VIFmax/VIFavg), and agreement metric (CCC).
- **Top models internal validation**: internal-validation metrics (Q^2, RMSE, MAE, and CCC) from three internal procedures: Leave-One-Out (LOO) cross-validation, repeated k-fold cross-validation, and Bootstrap (.632+) resampling validation. In this Models table, CCC replaces the residual standard-error column s for each procedure; the corresponding s values remain available in the Summary tab and in the models CSV file.
- **Top models external validation**: external prediction metrics Q^2_{F1}, Q^2_{F2}, Q^2_{F3}, RMSE(ext), MAE(ext), CCC(ext), and CWC.

Metric definitions are provided in Section 10.2. All three tables update automatically and remain synchronized with the Validation, Diagnostics, Visualization, Summary, Prediction, and Desktop tabs once a model set is available, either after a run or when loading saved results.

4.3 Validation tab

Configures internal and external validation procedures for the chosen models.

- **Model selection**: choose *Top*, *Multiple* (a user-defined range such as `1-5,7,9`), or *All* models.

- **Internal validation**: perform Leave-One-Out (LOO), repeated k-fold (*Folds* and *Repeats*), and/or Bootstrap .632+ (*Resamples*, minimum 200).
- **Hide controls**: each internal method (LOO, k-fold, Bootstrap .632+) includes a *Hide* option to suppress that method from internal-validation displays in Models and Summary while preserving the computed results in models file.
- **External validation**: select the external dataset file and delimiter; the target column must match that of the training set. When split mode is active (Random/Manual), external validation uses the split test dataset directly.

Validation results are appended automatically to the Models tab upon completion.

4.4 Diagnostics tab

Provides per-observation diagnostics and the correlation matrix for the active model.

- Model selector with status and dataset filter (*Training only*, *Training and Test*, or *Test only*); holdout status propagates automatically.
- Observation diagnostics table with *Observation*, *Set*, *Observed*, *Predicted*, *Residual*, *Std. residual*, *Z value*, *Leverage*, *Mahalanobis*, and *Cook*. Values above leverage and Mahalanobis thresholds are marked with $*$.
- LOO view toggle in the Dataset panel to display LOO columns when needed; export always includes the complete diagnostics dataset.
- Variable correlation matrix (lower triangle) with a toggle to include the dependent variable when available, plus CSV export.

4.5 Visualization tab

- Model selector: displays model availability and allows filtering by dataset scope (*Training only*, *Training and Test*, or *Test only*). Supports per-model Y-randomization runs and inserts the resulting charts into the chart list upon completion.
- Chart list: includes the standard regression diagnostics and the PCA chart *PC2 vs PC1* with 95% confidence ellipses, unified legend behavior, and autoscaling that accounts for points and ellipses.
- Plot controls: legend toggle with preset positions, labeling modes, interactive labels (left-click to add/drag, right-click to clear), identity line (enabled by default in compatible charts), optional linear fit (Predicted vs Observed), h^* threshold color for influence plots, grid toggle, synchronized axes, manual X/Y limits, and editable axis labels with adjustable font size. Options persist per chart type.

4.6 Summary tab

- Model selector with status plus a *Use recommended covariance* toggle.

- Overview: regression type, chosen covariance, search method, iterations per seed, target metric, models explored/found/reported (with threshold), significance level, observation counts, dependent variable, predictor list, k-fold plan, CPU times, clip-prediction bounds, and Y-randomization status.
- Covariance diagnostics (when the recommendation is enabled): recommended/applied type, Breusch–Pagan and White p-values, leverage threshold, maximum leverage, high-leverage count, and notes that triggered the robust choice.
- Coefficients and equation: coefficient table with standard errors, t, p (p-values), confidence intervals, and variable contribution percentages; equations rendered with and without errors.
- Metrics including training performance; Q^2, RMSE, MAE, s, and CCC for internal validation based on leave-one-out, repeated k-fold cross-validation, and Bootstrap (.632+); plus external validation, including external robustness and interval-based metrics (R^2_m, ΔR^2_m, r_s, C-index, CP, NMPIW, CWC, ER; definitions in Section 10.2, *External validation metrics*). Unlike the Models table, Summary retains s alongside CCC for all three internal-validation procedures.
- Applicability-domain and influential-observation sections with LD (leverage-based domain), MD (Mahalanobis-distance domain), and IQR (interquartile-range domain) blocks, threshold-aware counts/IDs, and a dedicated Test-set regional error analysis block.
- Training additional metrics: global model significance (F-statistic and its p-value), likelihood-based criteria (log-likelihood, AIC, BIC), residual diagnostics (Durbin–Watson, skewness, kurtosis, Jarque–Bera and Omnibus tests with associated probabilities), and numerical stability indicators (condition number); definitions in Section 10.2.

4.7 Prediction tab

- Dedicated prediction workspace for loading a CSV dataset, choosing delimiter, and selecting non-variable columns for table display.
- Two prediction modes: *Single model* and *Multiple models*, with automatic refresh when model options change.
- Multi-model output with applicability-domain interpretation, including *Only inside* filtering before averaging when enabled.
- Prediction table with sortable headers, fixed-height scrolling view, AD columns (LD/MD/IQR), and robust handling of missing predictor values (rows with missing entries are shown as unavailable).
- Plot previews include *View density histogram* (prediction-value distribution), *View RSD histogram* (residual spread), *View predictor count* (predictor-frequency ranking across selected models), and *View prediction PCA* (PC2 vs. PC1 projection using model predictors). All previews provide interactive plot controls/options, support CSV data export, and allow chart export at 300 dpi.

4.8 Desktop tab (Variable Explorer)

- Inputs: browse/load the CSV and an optional results file (unlocks model-aware options) with a readiness status line.
- Variable scope and axes: choose *All variables* or *Only variables of model* (enter an ID), then select X/Y axes. A *Histogram* toggle switches to histogram mode.
- *Predictor keys*: Open a dedicated window for multi-model predictor-frequency analysis. Select model IDs (single values and ranges), generate a ranked bar chart of predictor occurrences, and open *Model rank* to score models by aggregated predictor-key frequency (with optional Top filtering by CCC(ext), CWC, or Q^2_{LOO} when metrics are available). Use *Save plot* to export the chart to PNG, TIFF, PDF, or SVG.
- Plot controls: marker size/color/style, legend with selectable positions, gridlines, linear fit with optional R^2 display, partial residuals (when a model is active), aspect ratios (1:1, 4:3, 16:9, 9:16), manual axis limits and tick steps, and a dialog for axis labels and font size. Interactive labels can be added/dragged with left-click and cleared with right-click. Use *Save current plot* to export PNG, TIFF, PDF, or SVG.
- Filter dataset: *Create filtered dataset* option creates a CSV containing observation/ID/non-variable columns plus predictors from the current model or a specified model range.

4.9 Help tab

Summarizes product information and support links. The About panel shows the MLR-X name, version, license notice, and tagline. Quick links open the homepage and contact email. The *Manual* button opens the official online PDF manual URL; *How to cite* opens a citation dialog with copy-to-clipboard and `.bib` download; additional buttons open the bug-report page and donation page.

5 Analysis flow and process handling

5.1 Before starting

1. Load and preview the dataset, confirming delimiter, target, exclusions, and split mode.
2. Check that no other run is in progress, then choose where to save results when prompted (defaults to `models.csv`; a custom path is allowed).

5.2 Starting a run

1. Click Run analysis. The app validates the configuration, reloads the dataset (including the split) and prepares the selected output file.
2. The status switches to Running, and previous results, caches, and downstream tabs (*Diagnostics*, *Visualization*, *Summary*, *Prediction*, and *Desktop*) are reset.
3. The progress bar and step counter are reset for the search and post-processing stages.

5.3 Progress and cancellation

1. Progress updates show search and post-processing status with running counts.
2. *Cancel* stops all workers, performs cleanup, and leaves controls ready for another attempt.

5.4 Completion and outputs

1. Upon completion, the process status is set to *Finished*, and the results are automatically written to the specified output path (`models.csv` by default). The *Validation*, *Diagnostics*, *Visualization*, *Summary*, *Prediction*, and *Desktop* tabs are then enabled and populated with the new results.
2. If no models satisfy the reporting threshold, downstream analysis tabs remain disabled and an informational dialog is displayed to notify the user.

6 Search algorithms

6.1 EPR-C3 (Expand–Perturb–Reduce–C3)

EPR-C3 is a deterministic, multi-start, constraint-aware neighborhood-search heuristic implemented in MLR-X for ordinary least-squares (OLS) multiple linear regression subset selection. It is intended for large or high-dimensional predictor spaces where exhaustive all-subsets enumeration becomes operationally impractical, while the final models must remain explicit, interpretable, and statistically admissible. The algorithm operates per seed using the target metric, maximum predictor count, correlation threshold, VIF threshold, coefficient-significance level, and reporting thresholds defined in Setup & Run. Each seed follows a deterministic refinement loop:

1. Initialize the active subset with the seed-defined starting predictors and compute the baseline OLS metric.
2. **Expand**: add the inactive predictor that produces the largest screened improvement in the target metric, provided that the maximum model size is not exceeded.
3. **Perturb**: perform one-for-one exchanges between active and inactive predictors to escape locally optimal subsets.
4. **Reduce**: test one-variable removals and accept a removal when the target score is maintained or improved, thereby favoring parsimony when simplification is beneficial.
5. **C3 refinement**: apply correlation cleanup, swap recovery, and VIF pruning. Correlation cleanup removes one predictor from pairs exceeding the correlation threshold; swap recovery revisits predictors removed during cleanup and tests replacement swaps; VIF pruning iteratively removes high-VIF predictors when the active subset violates the VIF threshold.
6. Store only non-duplicated candidate models that satisfy the configured admissibility criteria, including model-size, target-metric, correlation, VIF, and coefficient-significance requirements.

7. Stop when no further gains are observed after repeated full cycles or when the configured iteration budget per seed is reached.

EPR-C3 runs are reproducible: for a fixed dataset, configuration options, seed list, and random state, the implementation follows the same search path and produces the same exported model set under the same numerical environment. In MLR-X, EPR-C3 should be understood as an implementation of the EPR-C3 method rather than as a method exclusive to the software. The methodological manuscript describing EPR-C3 is currently under peer review; the publication venue is intentionally not specified in this manual. By avoiding full combinatorial enumeration and instead exploring the model space through deterministic seeded local refinement, EPR-C3 provides an efficient alternative for large-scale and high-dimensional regression problems where exhaustive search is no longer practical.

6.2 All-subsets (traditional exhaustive search)

The all-subsets method (or best-subsets regression) is an exhaustive search strategy. It operates by explicitly enumerating and evaluating every possible combination of predictors, up to a user-defined limit. Because the number of possible models grows combinatorially with the number of predictors, this method quickly becomes computationally infeasible.

Prior to model fitting, predictors are pre-filtered using the correlation threshold to remove highly collinear variables. During evaluation, variance inflation factor (VIF) constraints are enforced to control multicollinearity. The procedure uses block-based processing with incremental flushing and top-K merge consolidation to keep memory usage stable while preserving consistent reporting criteria.

Because the computational cost increases exponentially with the number of predictors, this exhaustive approach is recommended only for small predictor spaces or low-dimensional subset limits, where complete enumeration remains computationally feasible.

Note: The GUI presents a warning dialog and the CLI emits structured warnings to inform whether execution should proceed; combination-count helpers estimate the scale of the search and provide guidance when the requested combinatorial space becomes computationally inefficient or not feasible.

7 Summary and reports

- Summary tab: *Export Summary...* writes a complete model dossier to a text file, including run metadata, search settings, coefficients, diagnostics, validation metrics (including the expanded metric set), and the chosen covariance setting for the currently selected model.
- Diagnostics tab: *Export CSV* saves the filtered observation diagnostics and the active correlation matrix (with or without the target) using semicolon-separated CSV.
- Visualization/plots: *Save current plot* exports the visible chart to PNG, TIFF, PDF, or SVG with high-DPI output when needed.

8 Command-line (CLI) usage

The CLI is available on Linux and in the macOS and Windows GUI+CLI packages. It enables execution of the complete MLR-X pipeline without launching the graphical interface and is recommended for automated, batch, or large-scale analyses.

8.1 Invocation

```
# Linux
./MLRX config.conf
./MLRX path/to/config.conf

# macOS GUI+CLI
./MLRX config.conf
./MLRX path/to/config.conf

# Windows GUI+CLI
MLRX.exe config.conf
MLRX.exe path\to\config.conf

# Python package mode
mlrx config.conf
mlrx path/to/config.conf
```

If additional or invalid arguments are supplied, the program displays the expected syntax and exits.

Paths containing spaces must be quoted, for example: `./MLRX"path/withspace/config.conf"`.

These command-line executions generate a models file in CSV format. The file can be subsequently loaded into the graphical interface to produce tables, charts, and diagnostic reports.

This workflow is recommended for most users, as the interface provides interactive visualization features, including chart customization, point labeling, and simplified data exploration.

8.2 Configuration file

Execution requires a plain-text configuration file organized into headed blocks.

Parameters are defined as key–value pairs. List-type parameters are selected by index using the `selected option` field.

Missing parameters automatically fall back to default values. A complete annotated example is provided in Section 13.

8.3 Optional arguments

The CLI accepts optional flags to control export behavior:

- `--version` prints the program version and exits.

- `--model <id>` selects a specific model for post-processing.
- `--outputs <items>` exports selected outputs. Valid tokens are: `diagnostics`, `visualization`, `pdf`, `png`, `tiff`, `svg`, `summary`, `rc`.
- `--noruns` skips model recomputation and regenerates outputs from an existing results file.
- `--onlyIV` executes internal validation only, using models from an existing results file at the configured output path.
- `--onlyEV` executes external validation only, using models from an existing results file at the configured output path.

Important token rules:

- `rc` is valid only immediately after `summary` (for `--outputs summary rc`).
- Figure-format tokens (`pdf`, `png`, `tiff`, `svg`) apply only when `visualization` is also requested.
- `--onlyIV` and `--onlyEV` skip model search and require that the configured results file already exists.

8.4 Output

Results are exported to a semicolon-separated CSV file containing normalized variable names and all training, internal, and external metrics.

If no model satisfies the target metric cutoff, an empty CSV file is generated.

By default, the file `models.csv` is written to the current working directory. The destination path can be changed in the configuration file.

When Optional arguments are used, the following additional outputs are produced:

8.4.1 Diagnostics — Tables

Diagnostics tables are written to the `Tables/` directory. Files use the prefix `model_#_`:

```
Tables/
  model_#_diagnostics.csv
  model_#_correlation.csv
  model_#_correlation_wdv.csv
```

The suffix `wdv` indicates the correlation table includes the dependent variable (with dependent variable).

8.4.2 Visualization — Charts

Charts are exported to the `Charts/` directory. All available charts are generated automatically.

```
model_#_predicted_vs_observed_train.pdf
model_#_predicted_vs_observed_test.pdf
model_#_y_randomization.pdf
```

```
model_#_residuals_train.pdf
...
```

8.4.3 Summary

Using –outputs summary, creates `model_#_summary.txt` file.

If `-outputs summary rc` is specified, the export enables *Use recommended covariance*.

8.5 CLI execution example

Linux example (model-specific exports):

```
./MLRX config.conf --model 1 --outputs diagnostics visualization pdf summary
```

This command exports charts, tables, and the summary for model 1.

If a models summary already exists, you can export charts, tables, and the summary for model 1 without re-running the analysis by adding `-noruns`:

```
./MLRX config.conf --model 1 --outputs diagnostics visualization pdf summary --noruns
```

To recalculate only internal validation metrics from an existing results file:

```
./MLRX config.conf --onlyIV
```

To recalculate only external validation metrics from an existing results file:

```
./MLRX config.conf --onlyEV
```

Note: In the configuration file, ensure the Output block uses the correct path and name for the models summary file.

9 Operational good practices

- Validate inputs before running: preview the CSV to confirm the delimiter, target column, non-variable columns, and exclusions. Apply the constant filter when near-constant predictors should be removed. If planning an external holdout, ensure identical headers, column order, and target definition across all files.
- Choose the search strategy by scope: use EPR-C3 for efficient exploration of large predictor spaces guided by deterministic seeded starts, and All Subsets when exhaustive coverage is required and longer runtimes are acceptable. In practice, EPR-C3 is preferred for large or high-dimensional designs, while All Subsets should be reserved for small datasets or confirmatory analyses; heed the combination-count guidance that flags inefficient All Subsets configurations.
- Configure EPR-C3 parameters thoughtfully: set the *Number of Seeds* to at least the number of predictors and adjust upward if premature convergence is observed—values above ∼1,000

seeds are recommended for robust searches. Choose a *Seed size* close to half of the maximum number of predictors to balance exploration depth and computational feasibility, and set a *Random state* whenever reproducibility matters.

- Leverage defaults, then refine: start from the default EPR-C3 values and adjust the *Target metric cutoff* and the *Export limit* (default 5,000) to control how many candidate models are retained and written to disk.
- Tune statistical thresholds before execution: define the *Significance level* to control predictor inclusion. Set the *Correlation threshold* (0.9 by default) and *Variance Inflation Factor (VIF) threshold*—preferably VIF < 5—to mitigate multicollinearity. Revisit these thresholds if the filtering is too permissive or overly restrictive.
- Pay close attention to software warnings and recommendations: during preparation and execution, review and act on guidance messages (for example, guardrails on insufficient seed counts, inefficient All Subsets combinations, or configuration consistency checks) to prevent avoidable errors and improve model-search robustness.
- Align parallelization with available hardware (CPU capacity).
- Validate comprehensively and interpret jointly: always run internal validation (LOO or k-fold) and, when available, external validation. Review the *Validation*, *Diagnostics*, *Visualization*, and *Summary* tabs together to assess model stability and reliability. Enable the recommended covariance option in the Summary tab when statistical inference is required.
- Apply prediction clipping cautiously: use *Clip predictions* only when the response variable has known physical or theoretical bounds, and ensure limits are consistent with the units and range of the training data.
- Preserve reproducibility: save all generated model CSVs and configuration files. These contain embedded metadata that allow future GUI or CLI sessions to reload and reproduce the exact run context.

10 Configuration Parameters, Metrics, and Diagnostic Charts

10.1 Key configuration parameters

- **Number of Seeds**: controls how many seed-defined starting points EPR-C3 explores; higher counts broaden coverage and reduce convergence to suboptimal local solutions.
- **Seed size**: sets the initial predictor count per seed; values near half the predictor limit (default) balance exploration and computation.
- **Random state (reproducibility)**: seed for reproducible EPR-C3 seed-list generation (default 42), used to build the same starting subsets across GUI, CLI, and configuration reloads.
- **Allow fewer than 1000 seeds**: A guardrail enforces a 1,000-seed minimum unless *Allow fewer than 1000 seeds* is enabled.

- **Set iterations/seed**: caps or adapts the exploration depth per seed through *auto*, *manual*, or *converge* modes.
- **Max predictors per model**: maximum number of predictors permitted in any candidate model.
- **Significance level**: primary p-value threshold used to accept or reject candidate models based on coefficient significance during model screening/refinement (models with coefficient p-values above the threshold are discarded). The same value is also used to set the prediction-interval confidence setting (for example, a significance level of 0.05 corresponds to a 95% prediction interval), which feeds the interval metrics reported in diagnostics/summary: *CP* (coverage probability), *NMPIW* (normalized mean prediction-interval width), and *CWC* (coverage-width criterion, where the coverage target is set by that confidence setting).
- **Correlation threshold**: maximum allowed absolute pairwise correlation between predictors; variables exceeding this limit are pruned to reduce redundancy.
- **VIF threshold**: exclusion criterion for candidate models exhibiting multicollinearity above the specified variance inflation factor, where high VIF values reflect an inflation of the estimated regression coefficients' variances due to linear dependence among predictors. In particular, VIF values in the range of 5–10 are commonly interpreted as indicative of problematic multicollinearity, while values exceeding 10 are associated with severely unstable and unreliable parameter estimates, as discussed and formalized by J. H. Kim, *Korean J. Anesthesiol.* 2019, 72, 558–569, and by M. O. Akinwande *et al.*, *Open J. Stat.* 2015, 5, 754–767.
- **Target metric**: performance measure used as the optimization objective (R^2, adjusted R^2, Q^2 (LOO), RMSE (LOO), or CCC).
- **Target metric cutoff**: minimum (or maximum, depending on the metric) training-set performance value a model must satisfy before being considered in the optimization stage.
- **Top models to report**: maximum number of best-performing models retained for final reporting; this value matches the CSV export limit.
- **Covariance estimator**: method used to estimate coefficient covariance matrices, where the *non-robust* estimator assumes homoskedastic errors under the classical ordinary least squares (OLS) framework. The *HC0* estimator replaces the constant error variance with squared OLS residuals, yielding a heteroskedasticity-consistent covariance matrix as originally proposed by H. White, *Econometrica* 1980, 48, 817–838. The *HC1* estimator applies a finite-sample degrees-of-freedom correction to HC0, while *HC2* down-weights high-leverage observations using the factor $1/(1-h_i)$, and *HC3* applies the stronger correction $1/(1-h_i)^2$, providing a jackknife-like adjustment and enhanced protection against leverage effects in finite samples, as formalized by J. G. MacKinnon *et al.*, *J. Econometrics* 1985, 29, 305–325. The use of HC3 is often recommended in practical applications due to its superior finite-sample performance and robustness to influential observations, particularly in small to moderate sample sizes, as discussed by J. S. Long *et al.*, *Am. Stat.* 2000, 54, 217–224.

- **Clip predictions**: lower and upper bounds applied to predicted values when outputs must respect known physical or empirical limits.
- **CPU cores to use**: number of worker processes requested for parallel execution.

10.2 Metric definitions

Unless noted otherwise, n_{train} denotes the number of training observations, n_{test} the external test size, p the number of predictors (excluding the intercept), y_i a training response, $\hat{y}_i$ its fitted value, and $e_i = y_i - \hat{y}_i$ the corresponding residual.

Training metrics The following statistics are computed on the training set:

- $\mathbf{R}^2$: coefficient of determination,

$$R^2 = 1 - \frac{\sum_{i=1}^{n_{\text{train}}} (y_i - \hat{y}_i)^2}{\sum_{i=1}^{n_{\text{train}}} (y_i - \bar{y}^{\text{train}})^2}.$$

- **Adj**–$\mathbf{R}^2$: adjusts for model size,

$$R^2_{\text{adj}} = 1 - (1 - R^2) \cdot \frac{n_{\text{train}} - 1}{n_{\text{train}} - p - 1}.$$

- **RMSE**: root mean squared error of the residuals,

$$\text{RMSE} = \sqrt{\frac{1}{n_{\text{train}}} \sum_{i=1}^{n_{\text{train}}} e_i^2}.$$

- **MAE**: mean absolute error,

$$\text{MAE} = \frac{1}{n_{\text{train}}} \sum_{i=1}^{n_{\text{train}}} |e_i|.$$

- $\boldsymbol{s}$: residual standard error with the degrees-of-freedom correction used in the code (falling back to n_{train} if $n_{\text{train}} - p - 1 \leq 0$),

$$s = \sqrt{\frac{\sum_{i=1}^{n_{\text{train}}} e_i^2}{n_{\text{train}} - p - 1}}.$$

- **VIFmax**: for each predictor x_j, the variance inflation factor is

$$\text{VIF}_j = \frac{1}{1 - R_j^2}$$

where R_j^2 comes from regressing x_j on the remaining predictors (with intercept). **VIFmax** is

$$\text{VIF}_{\max} = \max_j \text{VIF}_j.$$

- **VIFavg**: arithmetic mean of predictor-wise variance inflation factors,

$$\mathrm{VIF}_{\mathrm{avg}} = \frac{1}{p}\sum_{j=1}^{p}\mathrm{VIF}_j.$$

- **CCC**: Lin's concordance correlation coefficient on training predictions,

$$\mathrm{CCC} = \frac{2\,\mathrm{cov}(y,\hat{y})}{\sigma_y^2 + \sigma_{\hat{y}}^2 + (\mu_y - \mu_{\hat{y}})^2}.$$

Training additional metrics The following additional statistics are reported for training diagnostics:

- **F-statistic**: global significance statistic for the regression model,

$$F = \frac{(\mathrm{TSS} - \mathrm{RSS})/p}{\mathrm{RSS}/(n_{\mathrm{train}} - p - 1)} = \frac{\left(R^2/p\right)}{\left(1 - R^2\right)/(n_{\mathrm{train}} - p - 1)},$$

  where

$$\mathrm{TSS} = \sum_{i=1}^{n_{\mathrm{train}}}(y_i - \bar{y}^{\mathrm{train}})^2, \qquad \mathrm{RSS} = \sum_{i=1}^{n_{\mathrm{train}}} e_i^2.$$

  See J. Neter *et al.*, *Applied Linear Statistical Models*, 4th ed., Irwin, Chicago, 1996.

- **Prob (F-statistic)**: upper-tail probability associated with the global F test,

$$\mathrm{Prob}(F) = 1 - F_{\mathcal{F}_{p,\, n_{\mathrm{train}}-p-1}}(F),$$

  where $F_{\mathcal{F}_{p,\, n_{\mathrm{train}}-p-1}}(\cdot)$ is the cumulative distribution function of the F distribution with p and $n_{\mathrm{train}} - p - 1$ degrees of freedom. See J. Neter *et al.*, *Applied Linear Statistical Models*, 4th ed., Irwin, Chicago, 1996.

- **Log-Likelihood**: Gaussian log-likelihood at the maximum-likelihood estimate,

$$\log L = -\frac{n_{\mathrm{train}}}{2}\left[\log(2\pi) + 1 + \log\left(\frac{\mathrm{RSS}}{n_{\mathrm{train}}}\right)\right].$$

  See G. A. F. Seber and A. J. Lee, *Linear Regression Analysis*, 2nd ed., Wiley, Hoboken, 2003.

- **AIC**: Akaike information criterion,

$$\mathrm{AIC} = -2\log L + 2k,$$

  where $k = p + 1$ is the number of fitted regression coefficients including the intercept. See H. Akaike, *IEEE Trans. Autom. Control* 1974, 19, 716–723.

- **BIC**: Bayesian information criterion,

$$\mathrm{BIC} = -2\log L + k\log(n_{\mathrm{train}}),$$

where $k = p + 1$. See G. Schwarz, *Ann. Stat.* 1978, 6, 461–464.

- **Omnibus**: D'Agostino–Pearson omnibus normality statistic for residuals,

$$K^2 = Z_{\text{skew}}^2 + Z_{\text{kurt}}^2,$$

where Z_{skew} and Z_{kurt} are the normalized transformations of residual skewness and kurtosis, respectively. See R. B. D'Agostino *et al.*, *Am. Stat.* 1990, 44, 316–321.

- **Prob(Omnibus)**: upper-tail probability for the omnibus normality test,

$$\text{Prob(Omnibus)} = 1 - F_{\chi_2^2}(K^2),$$

where $F_{\chi_2^2}(\cdot)$ is the cumulative distribution function of the chi-squared distribution with 2 degrees of freedom. See R. B. D'Agostino *et al.*, *Am. Stat.* 1990, 44, 316–321.

- **Skew**: sample skewness of the residuals,

$$\text{Skew} = \frac{1}{n_{\text{train}}} \sum_{i=1}^{n_{\text{train}}} \left(\frac{e_i - \bar{e}}{s_e} \right)^3,$$

where

$$\bar{e} = \frac{1}{n_{\text{train}}} \sum_{i=1}^{n_{\text{train}}} e_i, \qquad s_e = \sqrt{\frac{1}{n_{\text{train}}} \sum_{i=1}^{n_{\text{train}}} (e_i - \bar{e})^2}.$$

See R. B. D'Agostino *et al.*, *Am. Stat.* 1990, 44, 316–321.

- **Kurtosis**: sample Pearson kurtosis of the residuals,

$$\text{Kurtosis} = \frac{1}{n_{\text{train}}} \sum_{i=1}^{n_{\text{train}}} \left(\frac{e_i - \bar{e}}{s_e} \right)^4.$$

A Gaussian residual distribution has Kurtosis = 3. See R. B. D'Agostino *et al.*, *Am. Stat.* 1990, 44, 316–321.

- **Durbin–Watson**: statistic for first-order residual autocorrelation,

$$\text{DW} = \frac{\sum_{i=2}^{n_{\text{train}}} (e_i - e_{i-1})^2}{\sum_{i=1}^{n_{\text{train}}} e_i^2}.$$

See J. Durbin and G. S. Watson, *Biometrika* 1950, 37, 409–428.

- **Jarque–Bera (JB)**: residual normality statistic based on skewness and kurtosis,

$$\text{JB} = n_{\text{train}} \left(\frac{\text{Skew}^2}{6} + \frac{(\text{Kurtosis} - 3)^2}{24} \right).$$

See C. M. Jarque and A. K. Bera, *Econ. Lett.* 1980, 6, 255–259.

- **Prob(JB)**: upper-tail probability for the Jarque–Bera test,

$$\text{Prob(JB)} = 1 - F_{\chi^2_2}(\text{JB}),$$

where $F_{\chi^2_2}(\cdot)$ is the cumulative distribution function of the chi-squared distribution with 2 degrees of freedom. See C. M. Jarque and A. K. Bera, *Econ. Lett.* 1980, 6, 255–259.

- **Cond. No**: condition number of the design matrix X,

$$\text{Cond. No} = \kappa(X) = \frac{\sigma_{\max}(X)}{\sigma_{\min}(X)},$$

where $\sigma_{\max}(X)$ and $\sigma_{\min}(X)$ are the largest and smallest singular values of X, respectively. See D. A. Belsley *et al.*, *Regression Diagnostics*, Wiley, New York, 1980.

- $|r|_{\max}$: maximum absolute pairwise Pearson correlation among predictors,

$$|r|_{\max} = \max_{j<k} |\text{corr}(x_j, x_k)| \,.$$

Internal validation metrics Internal validation is performed using leave-one-out (LOO), repeated k-fold cross-validation, and/or Bootstrap (.632+). For LOO, the prediction for observation i is computed as $\hat{y}_i^{(-i)} = y_i - e_i/(1-h_i)$, where h_i is the leverage, leading to LOO residuals $e_i^{\text{LOO}} = y_i - \hat{y}_i^{(-i)}$. The reported LOO metrics are

$$Q^2_{\text{LOO}} = 1 - \frac{\sum_i (e_i^{\text{LOO}})^2}{\sum_i (y_i - \bar{y}^{\text{train}})^2}, \quad \text{RMSE}_{\text{LOO}} = \sqrt{\frac{1}{n_{\text{train}}} \sum_i (e_i^{\text{LOO}})^2},$$

$$\text{MAE}_{\text{LOO}} = \frac{1}{n_{\text{train}}} \sum_i |e_i^{\text{LOO}}|, \quad \textit{and} \quad s_{\text{LOO}} = \sqrt{\frac{\sum_i (e_i^{\text{LOO}})^2}{n_{\text{train}} - p - 1}}$$

following the same degrees-of-freedom rule as s above. The k-fold versions use the corresponding out-of-fold residuals in place of e_i^{LOO} and are averaged across all folds/repeats.

Lin's concordance correlation coefficient is also calculated from the observed and validation-predicted responses:

$$\text{CCC}(y, \hat{y}) = \frac{2\,\text{cov}(y, \hat{y})}{\sigma_y^2 + \sigma_{\hat{y}}^2 + (\mu_y - \mu_{\hat{y}})^2}.$$

For LOO, the complete vector of leave-one-out predictions is used, so that

$$\text{CCC}_{\text{LOO}} = \text{CCC}\big(y, \hat{y}^{\text{LOO}}\big) \,.$$

For repeated k-fold validation, the out-of-fold predictions from all folds are assembled into one complete prediction vector for each repeat r. The reported value is the mean concordance over the R repeats:

$$\text{CCC}_{k\text{-fold}} = \frac{1}{R} \sum_{r=1}^{R} \text{CCC}\big(y, \hat{y}^{\text{OOF},r}\big) \,.$$

For Bootstrap (.632+), each resample b draws n_{train} observations with replacement (in-bag set), and predictions are evaluated both on in-bag and out-of-bag (OOB) observations. For a generic error measure err (MSE or MAE), the implementation computes

$$R_b = \frac{\text{err}_{\text{OOB},b} - \text{err}_{\text{resub},b}}{\gamma_b - \text{err}_{\text{resub},b}}, \qquad R_b \leftarrow \text{clip}(R_b, 0, 1),$$

with fallback $R_b = 1$ when $\gamma_b - \text{err}_{\text{resub},b} \leq 0$, and then

$$w_b = \frac{0.632}{1 - 0.368 R_b}, \qquad \text{err}_{.632+,b} = (1 - w_b)\,\text{err}_{\text{resub},b} + w_b\,\text{err}_{\text{OOB},b}.$$

The same MSE-derived weight w_b combines the in-bag and OOB concordance values for each valid resample:

$$\text{CCC}_{.632+,b} = (1 - w_b)\,\text{CCC}_{\text{resub},b} + w_b\,\text{CCC}_{\text{OOB},b}, \qquad \text{CCC}_{\text{BS}} = \frac{1}{B}\sum_{b=1}^{B} \text{CCC}_{.632+,b}.$$

The final Bootstrap (.632+) errors are the averages across valid resamples:

$$\text{MSE}_{\text{BS}} = \frac{1}{B}\sum_{b=1}^{B} \text{MSE}_{.632+,b}, \qquad \text{MAE}_{\text{BS}} = \frac{1}{B}\sum_{b=1}^{B} \text{MAE}_{.632+,b}.$$

From these, the reported internal Bootstrap metrics are

$$\text{RMSE}_{\text{BS}} = \sqrt{\text{MSE}_{\text{BS}}}, \qquad s_{\text{BS}} = \sqrt{\text{MSE}_{\text{BS}} \cdot \frac{n_{\text{train}}}{n_{\text{train}} - p - 1}},$$

and

$$Q^2_{\text{BS}} = 1 - \frac{\text{MSE}_{\text{BS}}}{\text{Var}(y^{\text{train}})}$$

where $\text{Var}(y^{\text{train}})$ is computed with denominator n_{train} (population variance), matching the code implementation.

External validation metrics External validation metrics are computed on the independent test set with residuals $e_i^{\text{test}} = y_i^{\text{test}} - \hat{y}_i^{\text{test}}$. They include Q^2_{F1}, Q^2_{F2}, Q^2_{F3},

$$\text{RMSE}_{\text{ext}} = \sqrt{\frac{1}{n_{\text{test}}}\sum_i (e_i^{\text{test}})^2}, \qquad \text{MAE}_{\text{ext}} = \frac{1}{n_{\text{test}}}\sum_i |e_i^{\text{test}}|,$$

and the external residual standard error

$$s_{\text{ext}} = \sqrt{\frac{\sum_i (e_i^{\text{test}})^2}{n_{\text{test}} - p - 1}},$$

with the same safe degrees-of-freedom fallback used in the code. The Q^2_{F1}, Q^2_{F2}, and Q^2_{F3} coefficients are based on the sum of squared prediction errors (SSE) and follow established QSAR validation conventions, differing in the choice of the reference variance used for normalization.

- $\mathbf{Q^2_{F1}}$: defined as

$$Q^2_{\mathrm{F1}} = 1 - \frac{\sum_{i=1}^{n_{\mathrm{test}}} \left(y_i^{\mathrm{test}} - \hat{y}_i^{\mathrm{test}}\right)^2}{\sum_{i=1}^{n_{\mathrm{test}}} \left(y_i^{\mathrm{test}} - \bar{y}^{\mathrm{train}}\right)^2}$$

where the denominator represents the variance of the external responses relative to the *training-set mean*. This formulation evaluates external predictivity with respect to the calibration reference learned during model fitting and penalizes deviations from the training mean, consistent with the external validation approach proposed by L. M. Shi *et al.*, *J. Chem. Inf. Comput. Sci.* 2001, 41, 186–195.

- $\mathbf{Q^2_{F2}}$: equivalent to the classical external coefficient of determination (R^2_{ext}),

$$Q^2_{\mathrm{F2}} = 1 - \frac{\sum_{i=1}^{n_{\mathrm{test}}} \left(y_i^{\mathrm{test}} - \hat{y}_i^{\mathrm{test}}\right)^2}{\sum_{i=1}^{n_{\mathrm{test}}} \left(y_i^{\mathrm{test}} - \bar{y}^{\mathrm{test}}\right)^2}$$

where the denominator corresponds to the variance of the external responses with respect to their own mean. This definition matches the external validation coefficient formalized and discussed by G. Schürmann *et al.*, *J. Chem. Inf. Model.* 2008, 48, 2140–2145.

- $\mathbf{Q^2_{F3}}$: compares the external mean squared error (MSE) with the normalized variance of the training set,

$$Q^2_{\mathrm{F3}} = 1 - \frac{\dfrac{1}{n_{\mathrm{test}}} \displaystyle\sum_{i=1}^{n_{\mathrm{test}}} \left(y_i^{\mathrm{test}} - \hat{y}_i^{\mathrm{test}}\right)^2}{\dfrac{1}{n_{\mathrm{train}}} \displaystyle\sum_{j=1}^{n_{\mathrm{train}}} \left(y_j^{\mathrm{train}} - \bar{y}^{\mathrm{train}}\right)^2}$$

where the numerator is the external MSE and the denominator is the variance of the training responses normalized by the number of training observations. This formulation, introduced by by V. Consonni *et al.*, *J. Chem. Inf. Model.* 2009, 49, 1669–1678, improves numerical stability for small or low-variance external sets.

- **Lin's CCC (external)**: Lin's concordance correlation coefficient computed on the external set,

$$\mathrm{CCC}_{\mathrm{ext}} = \frac{2\,\mathrm{cov}(y^{\mathrm{test}}, \hat{y}^{\mathrm{test}})}{\sigma^2_{y^{\mathrm{test}}} + \sigma^2_{\hat{y}^{\mathrm{test}}} + \left(\mu_{y^{\mathrm{test}}} - \mu_{\hat{y}^{\mathrm{test}}}\right)^2}.$$

See I. Lawrence and K. Lin, *Biometrics* 1989, 45, 255–268.

- **Roy's R^2_m family**: Roy's external validation metrics based on the standard coefficient of determination and origin-constrained regressions,

$$R^2_m = R^2 \left(1 - \sqrt{\left|R^2 - r_0^2\right|}\right), \qquad \Delta R^2_m = \left|R^2_m - R^{2\,\prime}_m\right|,$$

where r_0^2 is the squared correlation coefficient obtained from the regression through the origin of predicted versus observed values, and $R^{2\,\prime}_m$ is obtained by exchanging the observed and predicted axes in the corresponding origin-constrained regression. See K. Roy *et al.*, *Chemom. Intell. Lab. Syst.* 2011, 107, 194-205.

- **Spearman rank correlation**: Spearman rank-order correlation between observed and predicted external values,

$$r_s = \mathrm{corr}_{\mathrm{Spearman}}(y^{\mathrm{test}}, \hat{y}^{\mathrm{test}}).$$

See C. Spearman, *Am. J. Psychol.* 1904, 15, 72–101.

- **C-index (Harrell concordance)**: proportion of concordant comparable pairs in the external set,

$$C\text{-index} = \frac{1}{N_{\mathrm{pairs}}} \sum_{(i,j)} \mathbb{I}[(y_i - y_j)(\hat{y}_i - \hat{y}_j) > 0],$$

where the sum runs over all comparable pairs and N_{pairs} is the total number of such pairs. See F. E. Harrell Jr. *et al.*, *Stat. Med.* 1996, 15(4), 361–387.

- **Prediction-interval metrics** (at nominal coverage level α): coverage probability, normalized mean prediction-interval width, and coverage-width criterion,

$$\mathrm{CP} = \frac{1}{n_{\mathrm{test}}} \sum_{i=1}^{n_{\mathrm{test}}} \mathbb{I}(L_i \le y_i \le U_i), \qquad \mathrm{NMPIW} = \frac{\frac{1}{n_{\mathrm{test}}} \sum_{i=1}^{n_{\mathrm{test}}} (U_i - L_i)}{\max(y^{\mathrm{test}}) - \min(y^{\mathrm{test}})}.$$

$$\mathrm{CWC} = \begin{cases} \mathrm{NMPIW}, & \mathrm{CP} \ge \alpha, \\ \mathrm{NMPIW}\,[1 + \exp(-50(\mathrm{CP} - \alpha))], & \mathrm{CP} < \alpha. \end{cases}$$

See H. Khosravi *et al.*, *IEEE Trans. Neural Netw.* 2011, 22, 1341–1356.

- **ER**: leverage-based external error ratio implemented in this software,

$$h^* = \frac{3(p+1)}{n_{\mathrm{train}}}, \qquad \mathrm{ER} = \frac{\mathrm{RMSE}_{\mathrm{out}}}{\mathrm{RMSE}_{\mathrm{in}}},$$

where $\mathrm{RMSE}_{\mathrm{out}}$ and $\mathrm{RMSE}_{\mathrm{in}}$ are computed on external observations with leverage greater than and less than or equal to h^*, respectively. If no observation satisfies $h_i > h^*$, then $\mathrm{ER} = 0$; if no observation satisfies $h_i \le h^*$, then ER = infty. This metric is a software-specific implementation motivated by the leverage-threshold criterion.

10.3 Interpreting key charts

Correlation heatmaps Correlation heatmaps visualize pairwise Pearson correlation coefficients between predictors. High absolute correlation values indicate potential multicollinearity, which is controlled using the user-defined correlation threshold during model search. See Belsley *et al.*, *Regression Diagnostics: Identifying Influential Data and Sources of Collinearity*, Wiley, 1980; and Dormann *et al.*, *Ecography*, 2013, 36, 27–46.

Predicted vs. observed These plots compare fitted values $\hat{y}_i$ with observed responses y_i. Proximity to the identity line indicates good calibration, while systematic deviations suggest bias, underfitting, or overfitting. See Golbraikh and Tropsha, *J. Comput. Aided Mol. Des.* 2003, 17, 241–253

Residuals vs. predicted / observed Residual plots display $e_i = y_i - \hat{y}_i$ as a function of predicted or observed values. A random scatter around zero supports the assumptions of linearity and homoskedasticity, whereas visible structure or fan-shaped patterns indicate model misspecification or non-constant variance. See Montgomery *et al.*, *Introduction to Linear Regression Analysis*, 5th ed., Wiley, 2012.

Scale–location (Spread–location) The scale–location plot, also known as the spread–location plot, displays $\sqrt{|r_i|}$ versus fitted values, where r_i are standardized residuals. A roughly horizontal band indicates homoscedasticity, whereas a systematic increase or decrease in spread across the fitted range suggests heteroskedasticity and potential violations of the constant variance assumption. See Fox and Weisberg, *An R Companion to Applied Regression*, 3rd ed., Sage, 2019.

Residual distribution Histograms or density plots of residuals illustrate the overall error distribution. Symmetry around zero and moderate tails are consistent with approximately normal errors, whereas skewness or heavy tails may signal outliers or non-normality. See Kutner *et al.*, *Applied Linear Statistical Models*, 5th ed., McGraw–Hill, 2005.

Q–Q residuals Quantile–quantile plots compare standardized residuals against theoretical normal quantiles. Deviations from the diagonal, particularly in the tails, indicate departures from normality. See Montgomery *et al.*, *Introduction to Linear Regression Analysis*, 5th ed., Wiley, 2012.

Leverage plots Leverage plots display the diagonal elements of the hat matrix,

$$h_i = \mathbf{x}_i^\top (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{x}_i,$$

where $\mathbf{x}_i$ is the predictor vector (including the intercept). Observations with leverage exceeding the threshold

$$h^* = \frac{3(p+1)}{n}$$

are considered structurally influential and may disproportionately affect model estimates. See Atkinson, *Plots, Transformations and Regression*, Clarendon Press, Oxford, 1985; and Cook, *Technometrics*, 2000, 42(1), 65-68.

Cook's distance vs. leverage Cook's distance combines leverage and residual magnitude to assess the global influence of each observation:

$$D_i = \frac{e_i^2}{(p+1)s^2} \frac{h_i}{(1-h_i)^2}.$$

Large values of D_i indicate observations whose removal would substantially alter the fitted regression coefficients. See Cook and Weisberg, *Residuals and Influence in Regression*, Chapman & Hall, 1982, and Cook, *Technometrics*, 2000, 42(1), 65-68.

Williams plot The Williams plot jointly displays standardized residuals

$$r_i = \frac{e_i}{s\sqrt{1-h_i}}$$

against leverage values h_i. Observations exceeding both the standardized residual limits (typically ± 3) and the leverage threshold h^* are considered highly influential and should be examined carefully for data quality issues, model misspecification, or domain-specific justification. See Schürmann *et al.*, *J. Chem. Inf. Model.*, 2008, 48, 2140–2145; and Gramatica, *QSAR & Combinatorial Science*, 2007, 26, 694–701.

Predicted vs. leverage This plot relates fitted values $\hat{y}_i$ to leverage h_i, facilitating the identification of influential observations across the response range, particularly those combining extreme predictions with high structural influence. See Cook, *Technometrics*, 1977, 19, 15–18; and Fox and Weisberg, *An R Companion to Applied Regression*, 3rd ed., Sage, 2019.

10.4 Applicability Domain (AD)

The Applicability Domain (AD) defines where predictions are considered structurally reliable. In MLR-X, AD is reported with three complementary criteria: **LD**, **MD**, and **IQR**. These labels are used consistently across Prediction, Diagnostics, and Summary outputs.

AD (LD): Leverage domain For a model with p predictors and n_{train} training observations:

$$h^* = \frac{3(p+1)}{n_{\text{train}}}.$$

Each sample is labeled as:

- **Low value (Inside)** if $h_i \leq 0.5\,h^*$,
- **Moderate value (Inside)** if $0.5\,h^* < h_i \leq h^*$,
- **High value (Outside)** if $h_i > h^*$.

AD (MD): Mahalanobis domain Using squared Mahalanobis distance D_i^2 in predictor space and p degrees of freedom:

$$\chi^2_{0.5,p} \quad \text{and} \quad \chi^2_{0.95,p}.$$

Each sample is labeled as:

- **Low value (Inside)** if $D_i^2 < \chi^2_{0.5,p}$,
- **Moderate value (Inside)** if $\chi^2_{0.5,p} \leq D_i^2 \leq \chi^2_{0.95,p}$,
- **High value (Outside)** if $D_i^2 > \chi^2_{0.95,p}$.

AD (IQR): Predictor-IQR domain For each predictor, MLR-X computes training intervals from Q_1, Q_3, and $\text{IQR} = Q_3 - Q_1$:

- **Central interval**: $[Q_1 - \text{IQR},\ Q_3 + \text{IQR}]$,

- **Mild interval**: $[Q_1 - 1.5\,\mathrm{IQR},\ Q_3 + 1.5\,\mathrm{IQR}]$.

A sample is labeled as:

- **Central value (Inside)** when all predictor values are inside central intervals.
- **Mild value (Inside)** when not all are central, but all are inside mild intervals.
- **Outside value (Outside)** when at least one predictor falls outside the mild interval.

Practical interpretation and notation

- **Inside + low residual error**: generally robust prediction.
- **Inside + high residual error**: possible response-space anomaly.
- **Outside + low residual error**: potentially usable with caution.
- **Outside + high residual error**: high-risk prediction.

In multiple-model Prediction mode, aggregated AD can be reported as **Inside**, **Outside**, or **Mixed**, depending on how model-level AD labels combine for the same sample. For reliable interpretation, review AD together with residual diagnostics (Williams/Cook/standardized residuals), Summary AD blocks, and external metrics such as ER (Section 10.2).

11 Troubleshooting and FAQ

- **The GUI does not open**: download the corresponding package again and ensure the binary is not blocked by antivirus.
- **CSV does not load**: verify path, permissions, and delimiter; use *Preview data*.
- **No models exported**: review the *Target metric cutoff* and *VIF threshold*.
- **External validation fails**: the target column must match and the CSV must contain the same predictors.
- **Empty plot or missing labels**: select a valid model/dataset or load a results file.
- **Unexpected CPU usage**: check *CPU cores to use*; adjust to match the desired workload.
- **CLI error with `rc`**: use `rc` only after `summary` (example: `-outputs summary rc`).
- **No figures exported from CLI**: include `visualization` in `-outputs`; format tokens alone (pdf/png/tiff/svg) are not enough.
- **Chart names do not match**: use the current chart names shown in the Visualization tab (for example, *Predicted vs Observed* and *Q-Q residuals*).

Report any persisting issues or bugs at https://github.com/jacksonalcazar/MLR-X/issues so they can be addressed promptly.

12 License, credits, and citation

- **License**: MLR-X 1.0 is *free software*, distributed under the GNU Affero General Public License v3.0 (AGPL-3.0). Full license details are available in `LICENSE.txt`, in the *Help* tab of the application, and on the project homepage.
- **Author and copyright**: Jackson J. Alcázar (2026).
- **Citation**: when MLR-X 1.0 is used to generate results, analyses, or figures reported, please cite the software as:
  Alcázar, Jackson J. (2026). "MLR-X 1.0 software. Available at: https://jacksonalcazar.github.io/MLR-X/".

13 Configuration file example

```
# MLR-X configuration file
# Generated by MLR-X version 1.0

--------
Dataset
--------
Dataset path = data.csv

# Delimiter options:
1) ;
2) ,
3) \t
4) |
Selected option = 1

# Dependent variable options:
1) Last column
2) First column
3) Second column
4) Third column
Selected option = 1

# Non-variable columns options:
1) None
2) First column
3) First and second column
4) First, second, and third column
Selected option = 2

Exclude constant columns = false
Constant threshold = 90.0
Excluded observation IDs = none

--------------
Data splitting
--------------
# Split mode options:
1) None (use full dataset)
2) Random (test size (%) = none)
3) Manual (training IDs = none / test IDs = none)
```

```
Selected option = 1

---------
Settings
---------
# Search method options:
1) All subsets (traditional)
2) EPR-C3
Selected option = 2

# Target metric options:
1) R2
2) adj-R2
3) Q2 (LOO)
4) RMSE (LOO)
5) CCC
Selected option = 1

# Covariance type options:
1) Non robust
2) HC0
3) HC1
4) HC2
5) HC3
Selected option = 1

# Search Constraints
Max predictors per model = 6
Significance level = 0.05
Correlation threshold = 0.9
VIF threshold = 5.0
Target metric cutoff = 0.8
Top models to export = 5000

# Parallelization
CPU cores to use = 4

# Seed settings for EPR-C3
Number of seeds = 1000
Seed size = default
Random state = default
Allow fewer than 1000 seeds = false

# Iterations per seed mode options:
1) Auto
2) Manual (Max iterations per seed = none)
3) Until converge
Selected option = 1

# Prediction clipping
Clip predictions = false
Clip lower bound = none
Clip upper bound = none

# Internal and external validation:
LOO = true
k-fold = true
Folds = 10
```

```
Repeats = 5
BS = true
B = 1000
External dataset path = ext_data.csv

-------
Output
-------
Output path = models.csv
```